

## Assignment 5: Huffman Coding DESIGN.pdf:

### Description of Program:

This program has an encoder as well as a decoder. The Huffman encoder will read in an input file, find the Huffman encoding of its contents, and use the encoding to compress the file. The Huffman decoder will read in a compressed input file and decompress it, expanding it back to its original, uncompressed size.

Files to be included in directory "asgn5" + pseudocode/structure:

- node.h: This file will contain the node ADT interface. This file will be provided.
- node.c: This file will contain the implementation of the node ADT.
  - Struct for Node has a pointer to left and right child, symbol, and frequency
  - node\_create() is the constructor for a node and sets node's symbol and frequency
  - Node\_delete is the destructor for the node
  - Node\_join joins the left and right child nodes, and returns a pointer to a created parent node. The parent node's left child will be left and its right child will be right. The parent node's symbol will be '\$' and its frequency the sum of its left child's frequency and its right child's frequency.
  - Node\_print is a debug function that prints the node

- pq.h: This file will contain the priority queue ADT interface. This file will be provided.
- pq.c: This file will contain the implementation of the priority queue ADT.
  - Pq\_create is the constructor for a priority queue. The priority queue's maximum capacity is specified by capacity.
  - Pq\_delete is the destructor for a priority queue. Sets the pointer to NULL after freeing the memory for a priority queue.
  - pq\_empty returns true if the priority queue is empty and false otherwise
  - pq\_full returns true if the priority queue is full and false otherwise
  - pq\_size returns the number of items currently in the priority queue.
  - enqueue() enqueues a node into the priority queue. Returns false if the priority queue is full prior to enqueueing the node and true otherwise to indicate the successful enqueueing of the node.
  - dequeue() dequeues a node from the priority queue, passing it back through the double pointer n. The node dequeued should have the highest priority over all the nodes in the priority queue. Returns false if the priority queue is empty prior to dequeuing a node and true otherwise to indicate the successful de-queueing of a node.
  - pq\_print() debug function to print the priority queue
- code.h: This file will contain the code ADT interface. This file will be provided.

- `code.c`: This file will contain the implementation of the code ADT.
  - Struct `code` has `top` and `bits[MAX_CODE_SIZE]`
  - `code_init()` This function will not require any dynamic memory allocation. Create a new Code on the stack, setting `top` to 0, and zeroing out the array of bits, `bits`. The initialized Code is then returned. No destructor.
  - `Code_size` Returns the size of the Code, which is exactly the number of bits pushed onto the Code.
  - `code_empty()` returns true if code is empty, false otherwise
  - `code_full()` Returns true if the Code is full and false otherwise. The maximum length of a code in bits is 256, which we have defined using the macro `ALPHABET`.
  - `Code_set_bit()` Sets the bit at index `i` in the Code, setting it to 1. If `i` is out of range, return false. Otherwise, return true to indicate success.
  - `Code_clr_bit()` Clears the bit at index `i` in the Code, clearing it to 0. If `i` is out of range, return false. Otherwise, return true to indicate success.
  - `Code_get_bit()` Gets the bit at index `i` in the Code. If `i` is out of range, or if bit `i` is equal to 0, return false. Return true if and only if bit `i` is equal to 1.

- `Code_push_bit()` Pushes a bit onto the Code. The value of the bit to push is given by `bit`. Returns false if the Code is full prior to pushing a bit and true otherwise to indicate the successful pushing of a bit.
- `Code_pop_bit()` Pops a bit off the Code. The value of the popped bit is passed back with the pointer `bit`. Returns false if the Code is empty prior to popping a bit and true otherwise to indicate the successful popping of a bit.
- `code_print()` debugging print function
- `io.h`: This file will contain the I/O module interface. This file will be provided.
- `io.c`: This file will contain the implementation of the I/O module.
  - `read_bytes()` wrapper function to loop calls to `read()` until we have either read all the bytes that were specified (`nbytes`) into the byte buffer `buf`, or there are no more bytes to read. The number of bytes that were read from the input file descriptor, `infile`, is returned.
  - `write_bytes()` loop until we have either written out all the bytes specified from the byte buffer `buf`, or no bytes were written. The number of bytes written out to the output file descriptor, `outfile`, is returned.
  - `read_bit()` function to read a single bit. maintain a static buffer of bytes and an index into the buffer that tracks which bit to return through the pointer `bit`. The buffer will store `BLOCK` number of bytes, where `BLOCK` is yet another macro defined in `defines.h`.

This function returns false if there are no more bits that can be read and true if there are still bits to read.

- write\_code() This function will also make use of a static buffer and an index. Each bit in the code c will be buffered into the buffer. The bits will be buffered starting from the 0th bit in c. When the buffer of BLOCK bytes is filled with bits, write the contents of the buffer to outfile.
- flush\_codes() write out any leftover, buffered bits. extra bits in the last byte are zeroed before flushing the codes.
- stack.h: This file will contain the stack ADT interface. This file will be provided.
- stack.c: This file will contain your implementation of the stack ADT.
  - Struct for stack has top, capacity, and Node \*\*items
  - stack\_create() The constructor for a stack. The maximum number of nodes the stack can hold is specified by capacity
  - stack\_delete() the destructor for a stack. Set the pointer to NULL after freeing memory allocated by the stack.
  - stack\_empty() returns true if the stack is empty and false otherwise.
  - stack\_full() returns true if the stack is full and false otherwise.
  - stack\_push() pushes a node onto the stack. Returns false if the stack is full prior to pushing the node and true otherwise to indicate the successful pushing of a node.

- `Stack_pop` pops a node off of the stack, passing it back through the double pointer `n`. Returns false if the stack is empty prior to popping a node and true otherwise to indicate the successful popping of a node.
- `stack_print()` debug function to print contents of the stack
- `huffman.h`: This file will contain the Huffman coding module interface
- `huffman.c`: This file will contain the implementation of the Huffman coding module interface.
- `build_tree()` constructs a Huffman tree given a computed histogram. The histogram will have ALPHABET indices, one index for each possible symbol. Returns the root node of the constructed tree.
  - Create a priority queue. For each symbol histogram where its frequency is greater than 0, create a corresponding Node and insert it into the priority queue.
  - While there are two or more nodes in the priority queue, dequeue two nodes. The first dequeued node will be the left child node. The second dequeued node will be the right child node. Join these nodes together using `node_join()` and enqueue the joined parent node. The frequency of the parent node is the sum of its left child's frequency and its right child's frequency.
  - Eventually, there will only be one node left in the priority queue. This node is the root of the constructed Huffman tree.

- `build_codes()`
  - Create a new Code `c` using `code_init()`. Starting at the root of the Huffman tree, perform a post-order traversal. If the current node is a leaf, the current code `c` represents the path to the node, and thus is the code for the node's symbol. Save this code into the code table. Else, the current node must be an interior node. Push a 0 to `c` and recurse down the left link. After you return from the left link, pop a bit from `c`, push a 1 to `c` and recurse down the right link. Pop a bit from `c` when you return from the right link.
- `dump_tree()`
  - Conducts a post-order traversal of the Huffman tree rooted at root, writing it to outfile. This should write an 'L' followed by the byte of the symbol for each leaf, and an 'I' for interior nodes.
- `rebuild_tree()`
  - The array containing the dumped tree will be referred to as `tree_dump`. The length of this array will be `nbytes`. A stack of nodes will be needed to reconstruct the tree. Iterate over the contents `tree_dump` from 0 to `nbytes`. If the element of the array is an 'L', then the next element will be the symbol for the leaf node. Use that symbol to create a new node with `node_create()`. Push the created node onto the stack. If the

element of the array is an 'I', then you have encountered an interior node. Pop the stack once to get the right child of the interior node, then pop again to get the left child of the interior node. Join the left and right nodes with `node_join()` and push the joined parent node on the stack. There will be one node left in the stack after you finish iterating over the contents `tree_dump`. This node is the root of the Huffman tree.

- `delete_tree()`
  - The destructor for a Huffman tree. This will require a post-order traversal of the tree to free all the nodes. Set the pointer to NULL after you are finished freeing all the allocated memory.
- `encode.c`: This file will contain the implementation of the Huffman encoder.
  - Parse command-line options with `getopt()`, Read through infile to construct a histogram. Histogram should be a simple array of 256 (ALPHABET) `uint64_ts`. Increment the count of element 0 and element 255 by one in the histogram. This is so that at the very minimum, the histogram will have two elements present. Do this regardless of what is read in. Construct the Huffman tree using a priority queue. This will be done using `build_tree()`. Construct a code table by traversing the Huffman tree. This will be done using `build_codes()`. The code table is a simple array of 256 (ALPHABET)



Codes. Construct a header. Write the constructed header to outfile.

Write the constructed Huffman tree to outfile using `dump_tree()`.

Starting at the beginning of infile, write the corresponding code for each symbol to outfile with `write_code()`. When finished with all the symbols, flush any remaining buffered codes with `flush_codes()`. Close infile and outfile.

- `decode.c`: This file will contain the implementation of the Huffman decoder.
  - Read in the header from infile and verify the magic number. If the magic number does not match `0xBEEFD00D` (defined as `MAGIC` in `defines.h`), then an invalid file was passed. Display a helpful error message and quit. The permissions field in the header indicates the permissions that outfile should be set to. Set the permissions using `fchmod()`. The size of the dumped tree is given by the `tree_size` field in the header. Read the dumped tree from infile into an array that is `tree_size` bytes long. Then, reconstruct the Huffman tree using `rebuild_tree()`. Read infile one bit at a time using `read_bit()`. Traversing down the tree one link at a time for each bit that is read. Begin at the root of the Huffman tree. If a bit of value 0 is read, then walk down to the left child of the current node. Else, if a bit of value 1 is read, then walk down to the right child of the current node. If at a leaf node, then write the leaf node's symbol to outfile. After writing the symbol, reset the current node back to the root of the tree. Repeat until the number of decoded symbols matches the original

file size, which is given by the `file_size` field in the header that was read from `infile`. Close `infile` and `outfile`.

- `defines.h`: This file will contain the macro definitions used throughout the assignment.
- `header.h`: This will contain the struct definition for a file header.
  - The header's magic number field, `magic`, should be set to the macro `MAGIC`, as defined in `defines.h`. This magic number identifies a file as one which has been compressed using the encoder. The header's permissions field stores the original permission bits of `infile`. Use `fstat()`. Set the permissions of `outfile` to match the permissions of `infile` using `fchmod()`. The header's `tree_size` field represents the number of bytes that make up the Huffman tree dump. This size is calculated as  $(3 \times \text{unique symbols}) - 1$ . The header's `file_size` field is the size in bytes of the file to compress, `infile`. Obtain this size through `fstat()` as well.
- `Makefile`
  - • `CC = clang` must be specified.
  - • `CFLAGS = -Wall -Wextra -Werror -Wpedantic` must be included.
  - • `make` should build the encoder and the decoder, as should `make all`.
  - • `make encode` should build just the encoder.
  - • `make decode` should build just the decoder.
  - • `make clean` must remove all files that are compiler generated.

- • make format should format all your source code, including the header files.
- README.md
  - In proper Markdown syntax.
  - Describe how to use your program and Makefile
  - Lists and explains any command-line options that program accepts.
  - Reports any false positives by scan-build
  - Notes down any known bugs or errors in this file as well for the graders.
- DESIGN.pdf
  - This document must be a proper PDF.
  - Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.